

✓ MIS780 Advanced AI For Business - Assessment 2 - T2 2025

Task 3 – Forecasting Air Quality Pollution (PM2.5 Concentration) with Time-series Data

Student Name: Leon Wilson

Student ID: 224142965

Table of Content

1. [Executive Summary](#)
2. [Data Preprocessing](#)
3. [Predictive Modeling](#)
4. [Experiments Report](#)
5. [Role of GenAI](#)

✓ 1. Executive Summary

Air pollution, particularly PM2.5 (particulate matter with diameter $\leq 2.5\mu\text{m}$), poses significant health risks and environmental challenges. Accurate forecasting of PM2.5 concentration is crucial for public health advisories, environmental policy-making, and urban planning initiatives.

In this task, a comprehensive multivariate hourly dataset spanning March 2013 to February 2017 was provided, including PM2.5 concentrations and critical air quality indicators such as CO, NO2, SO2, and wind speed. The primary objective was to develop a robust Recurrent Neural Network (RNN) model capable of forecasting PM2.5 levels for a 24-hour horizon, with data prior to 2016 used for training and data from January 2016 reserved for testing.

Multiple experiments were conducted with different RNN architectures including simple LSTM, stacked LSTM, Bidirectional LSTM, and GRU networks. Each architecture was evaluated using mean absolute error (MAE) across the 24-hour forecast horizon. Results indicated that a simple single-layer LSTM provided the best predictive performance (MAE ≈ 43.14), outperforming more complex architectures and GRU-based models.

The findings highlight that, for noisy multivariate hourly PM2.5 data, model simplicity combined with appropriate feature selection and sequence length is more effective than deeper or bidirectional architectures. The trained LSTM model can be applied to predict future PM2.5 concentrations, supporting early health advisories, mitigation strategies, and informed urban planning. Future improvements could involve incorporating additional meteorological features, tuning sequence lengths, or experimenting with ensemble methods for enhanced forecasting accuracy.

✓ 2. Data Preprocessing

```
# Importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import random

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout
from tensorflow.keras.layers import RepeatVector, TimeDistributed
from tensorflow.keras.optimizers import SGD
from tensorflow.random import set_seed
```

```
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
```

```
# Set seeds
seed = 42
os.environ['PYTHONHASHSEED'] = str(seed)
random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
```

```
air_data = pd.read_csv(
    "/content/Task 3 Dataset Air Quality.csv",
    parse_dates=['datetime'],
    index_col='datetime',
    dayfirst=True
)
print(air_data.head())
```

```

datetime      PM2.5  SO2   NO2   CO   O3  TEMP   PRES  DEWP  WSPM
2013-03-01 00:00:00    5.0  4.0  12.0  200  85.0  -0.5  1024.5 -21.4   5.7
2013-03-01 01:00:00    8.0  6.0  14.0  200  84.0  -0.7  1025.1 -22.1   3.9
2013-03-01 02:00:00    3.0  5.0  14.0  200  83.0  -1.2  1025.3 -24.6   5.3
2013-03-01 03:00:00    5.0  5.0  14.0  200  84.0  -1.4  1026.2 -25.5   4.9
2013-03-01 04:00:00    5.0  6.0  21.0  200  77.0  -1.9  1027.1 -24.5   3.2
```

```
air_data.isnull().sum()
```

```

0
PM2.5    0
SO2      0
NO2    692
CO       0
O3    506
TEMP     0
PRES     0
DEWP     0
WSPM     0

dtype: int64
```

```
air_data.describe()
```

```

count    35064.000000  35064.000000  34372.000000  35064.000000  34558.000000  35064.000000  35064.000000  35064.000000  35064.000000  35064.000000
mean      84.518994    18.679584    58.097172   1320.073266    58.534682    13.672020   1012.547222     2.447850
std      85.482426    24.125895    36.297740   1223.774709    58.401448    11.455204    10.263134    13.806763
min       2.000000     0.570000     2.000000    100.000000     0.214200   -16.800000    987.100000   -35.300000
25%      23.000000     3.000000    29.000000    500.000000    10.000000     3.100000   1004.000000    -8.800000
50%      60.000000     9.000000    51.000000   1000.000000    45.000000    14.600000   1012.200000     3.000000
75%     114.000000    23.000000    80.000000   1600.000000    84.000000    23.500000   1020.900000    15.000000
max     844.000000   257.000000   273.000000  10000.000000   390.000000    41.100000   1042.000000    28.800000
```

```
# Impute missing values for NO2 and O3 directly
air_data['NO2'] = air_data['NO2'].interpolate(method='time')
air_data['NO2'] = air_data['NO2'].ffill().bfill()

air_data['O3'] = air_data['O3'].interpolate(method='time')
```

```
air_data['O3'] = air_data['O3'].ffill().bfill()
```

```
print(air_data.isnull().sum())
```

```
PM2.5    0
SO2       0
NO2       0
CO        0
O3        0
TEMP      0
PRES      0
DEWP      0
WSPM      0
dtype: int64
```

```
# Compute the correlation matrix (using Pearson correlation by default)
```

```
corr_matrix = air_data.corr()
```

```
# Display the correlation matrix as a heatmap
```

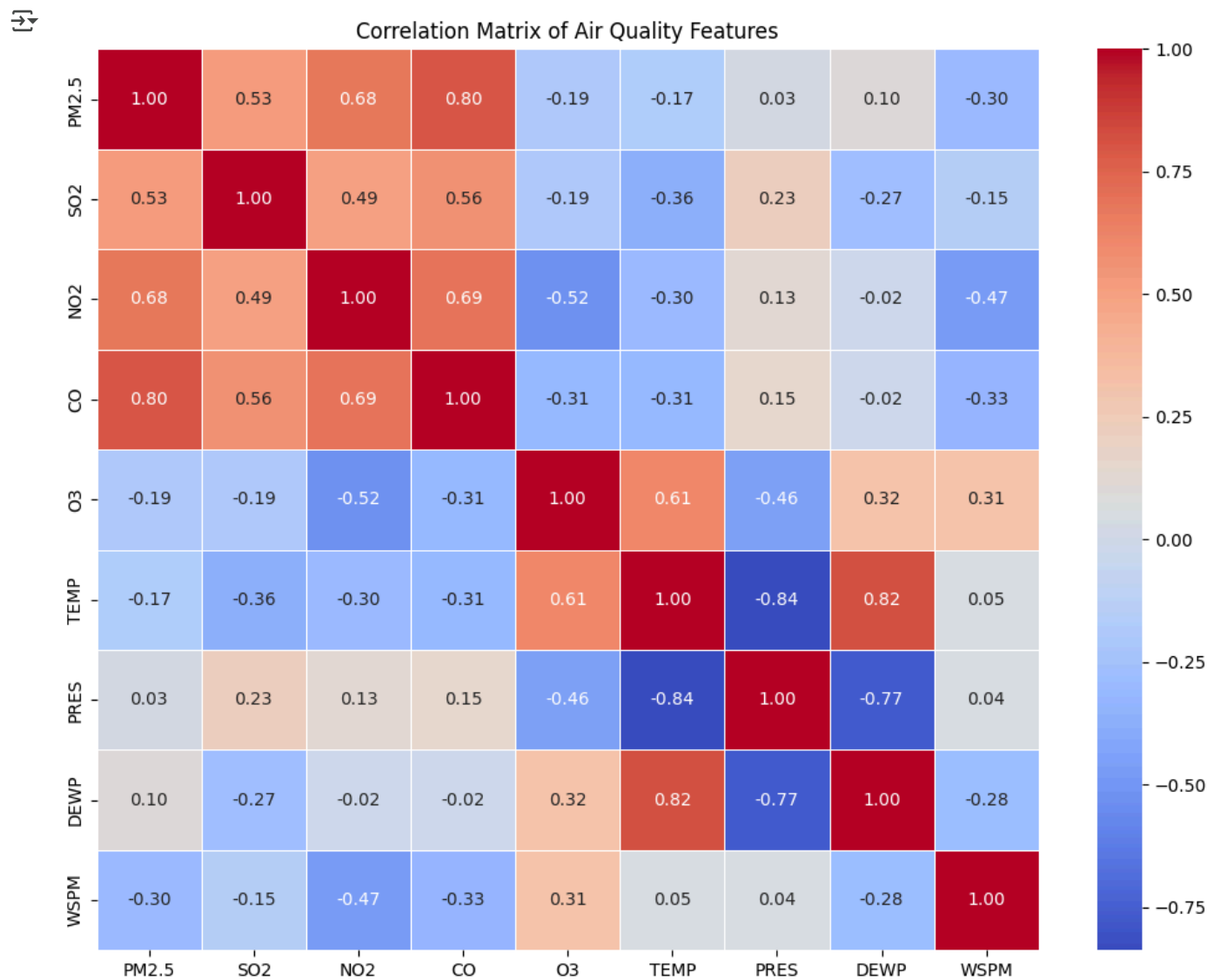
```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', linewidths=0.5)
```

```
plt.title('Correlation Matrix of Air Quality Features')
```

```
plt.tight_layout()
```

```
plt.show()
```



```
fig, axs = plt.subplots(4, 1, figsize=(12, 14))
```

```
axs[0].plot(air_data['PM2.5'], color='tab:green')
```

```
axs[0].set_title('PM2.5 Concentration')
```

```
axs[1].plot(air_data['NO2'], color='tab:blue')
```

```
axs[1].set_title('NO2 Concentration')
```

```

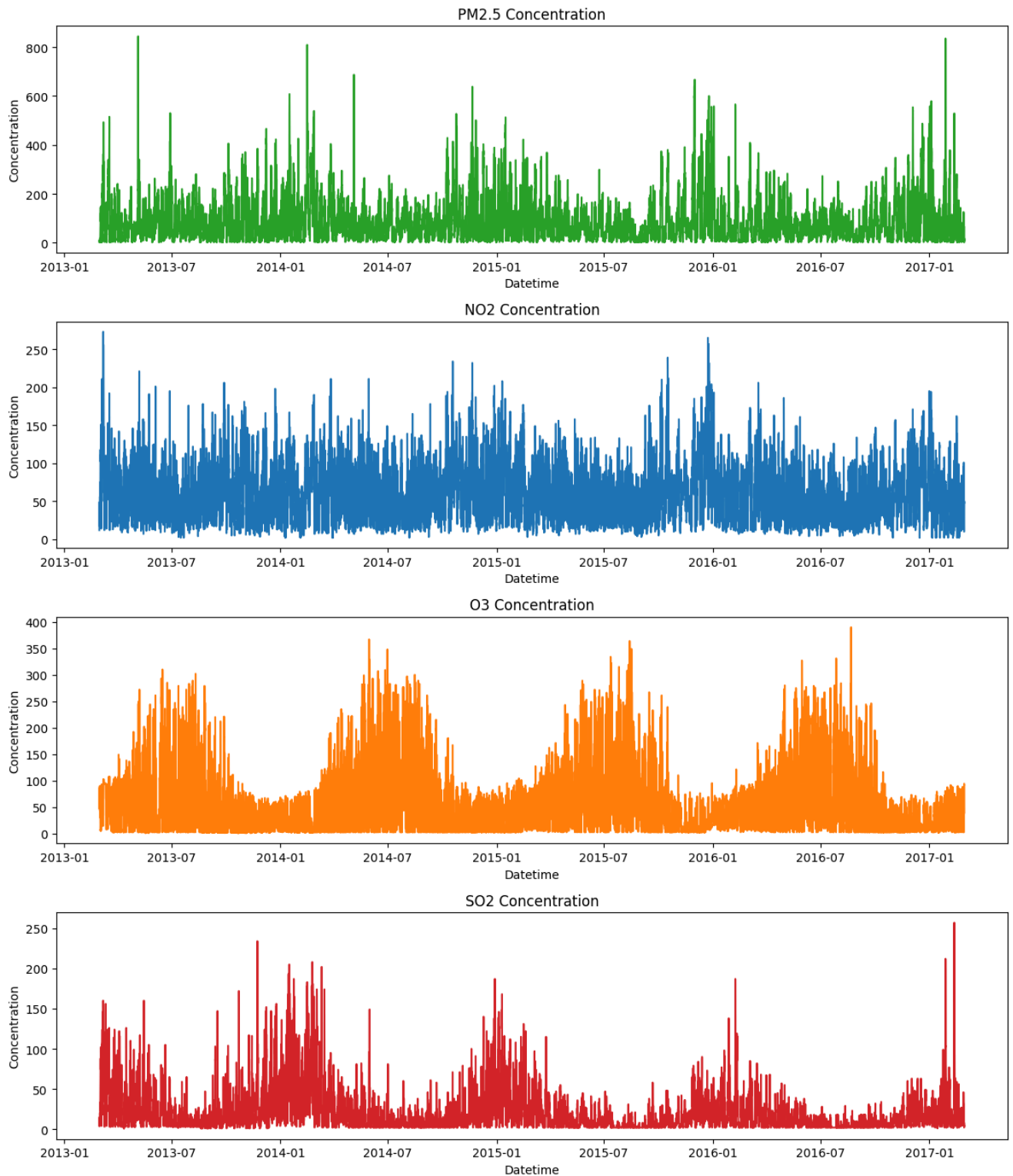
axs[2].plot(air_data['O3'], color='tab:orange')
axs[2].set_title('O3 Concentration')

axs[3].plot(air_data['SO2'], color='tab:red')
axs[3].set_title('SO2 Concentration')

for ax in axs:
    ax.set_xlabel('Datetime')
    ax.set_ylabel('Concentration')

plt.tight_layout()
plt.show()

```



✓ 3. Predictive Modeling

Multivariate Models

```
feature_cols = ['CO', 'NO2', 'SO2', 'WSPM'] #based on the correlation with PM2.5
target_col = 'PM2.5'
selected_cols = feature_cols + [target_col]
dataset = air_data[selected_cols]
```

```
# 1. Train-test split: before 2016 = train, January 2016 = test
train = dataset.loc[:'2015-12-31 23:00:00']
test = dataset.loc['2016-01-01 00:00:00':'2017-02-28 23:00:00']
```

```
training_set_shape = train.shape
test_set_shape = test.shape
print('training_set shape:', training_set_shape)
print('test_set shape:', test_set_shape)
```

```
➦ training_set shape: (24864, 5)
test_set shape: (10200, 5)
```

```
sc = MinMaxScaler(feature_range=(0, 1))
training_set_scaled = sc.fit_transform(train.values) # scale all 5 columns separately
test_set_scaled = sc.transform(test.values) # scale test set with same scaler
print('training_set_scaled shape:', training_set_scaled.shape)
print('test_set_scaled shape:', test_set_scaled.shape)
```

```
➦ training_set_scaled shape: (24864, 5)
test_set_scaled shape: (10200, 5)
```

```
#The function split_sequence is used to convert time series data into supervised learning samples
def split_sequence(sequence, n_steps, forecasting_horizon, y_index):
    X, y = [], []
    for i in range(len(sequence)):
        end_ix = i + n_steps
        if end_ix > len(sequence) - forecasting_horizon:
            break
        seq_x = sequence[i:end_ix]
        seq_y = sequence[end_ix:end_ix+forecasting_horizon, y_index]
        X.append(seq_x)
        y.append(seq_y)
    return np.array(X), np.array(y)
```

```
n_steps = 48 #previous 48 hours data
forecasting_horizon = 24 #next 24 hours
features = 5
y_index = 4 # PM2.5 column index
```

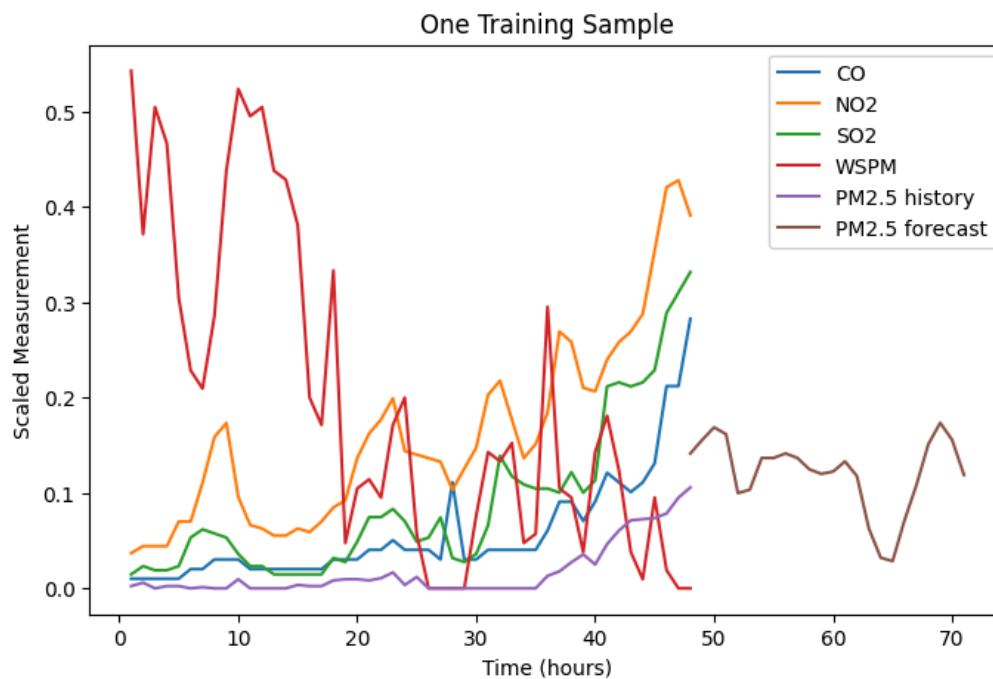
```
X_train, y_train = split_sequence(training_set_scaled, n_steps, forecasting_horizon, y_index)
y_train = y_train.reshape(y_train.shape[0], y_train.shape[1], 1)
```

```
print('X_train shape:', X_train.shape) # (samples, 48, 5)
print('y_train shape:', y_train.shape) # (samples, 24, 1)
```

```
➦ X_train shape: (24793, 48, 5)
y_train shape: (24793, 24, 1)
```

```
# Visualize one training sample
plt.figure(figsize=(8, 5))
plt.plot(np.arange(1, n_steps+1, 1), X_train[0,:,0]) # CO
plt.plot(np.arange(1, n_steps+1, 1), X_train[0,:,1]) # NO2
plt.plot(np.arange(1, n_steps+1, 1), X_train[0,:,2]) # SO2
plt.plot(np.arange(1, n_steps+1, 1), X_train[0,:,3]) # WSPM
plt.plot(np.arange(1, n_steps+1, 1), X_train[0,:,4]) # PM2.5 history
plt.plot(np.arange(n_steps, n_steps+forecasting_horizon, 1), y_train[0,:,0]) # PM2.5 forecast
```

```
plt.title('One Training Sample')
plt.ylabel('Scaled Measurement')
plt.xlabel('Time (hours)')
plt.legend(['CO', 'NO2', 'SO2', 'WSPM', 'PM2.5 history', 'PM2.5 forecast'], loc='upper right')
plt.show()
```



```
# 1. Define LSTM model
model_lstm = Sequential()
model_lstm.add(LSTM(units=100, activation='tanh', input_shape=(n_steps, features)))
model_lstm.add(Dense(units=forecasting_horizon)) # output 24 hours
model_lstm.compile(optimizer='RMSprop', loss='mse', metrics=['mae'])
model_lstm.summary()
```



/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`in`
super().__init__(**kwargs)

Model: "sequential_18"

Layer (type)	Output Shape	Param #
lstm_15 (LSTM)	(None, 100)	42,400
dense_22 (Dense)	(None, 24)	2,424

Total params: 44,824 (175.09 KB)
Trainable params: 44,824 (175.09 KB)
Non-trainable params: 0 (0.00 B)

```
#stacked LSTM Model
model_stacked_lstm = Sequential()
model_stacked_lstm.add(LSTM(128, activation='tanh', return_sequences=True, input_shape=(n_steps, features)))
model_stacked_lstm.add(LSTM(64, activation='tanh'))
model_stacked_lstm.add(Dense(forecasting_horizon))

model_stacked_lstm.compile(optimizer=Adam(0.001), loss='mse', metrics=['mae'])
model_stacked_lstm.summary()
```

Model: "sequential_19"

Layer (type)	Output Shape	Param #
lstm_16 (LSTM)	(None, 48, 128)	68,608
lstm_17 (LSTM)	(None, 64)	49,408
dense_23 (Dense)	(None, 24)	1,560

Total params: 119,576 (467.09 KB)
Trainable params: 119,576 (467.09 KB)
Non-trainable params: 0 (0.00 B)

```
from tensorflow.keras.layers import Bidirectional
#Bidirectional LSTM
model_bi_lstm = Sequential()
model_bi_lstm.add(Bidirectional(LSTM(100, activation='tanh'), input_shape=(n_steps, features)))
model_bi_lstm.add(Dense(64, activation='relu'))
model_bi_lstm.add(Dense(forecasting_horizon))
model_bi_lstm.compile(optimizer=Adam(0.001), loss='mse', metrics=['mae'])
model_bi_lstm.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/bidirectional.py:107: UserWarning: Do not pass an `input\_shape` argument to the Bidirectional layer. Use the Bidirectional layer's `input\_shape` attribute instead.
super().__init__(**kwargs)

Model: "sequential_20"

Layer (type)	Output Shape	Param #
bidirectional_2 (Bidirectional)	(None, 200)	84,800
dense_24 (Dense)	(None, 64)	12,864
dense_25 (Dense)	(None, 24)	1,560

Total params: 99,224 (387.59 KB)
Trainable params: 99,224 (387.59 KB)
Non-trainable params: 0 (0.00 B)

```
from tensorflow.keras.layers import GRU
#Gru Model
model_gru = Sequential()
model_gru.add(GRU(150, activation='tanh', input_shape=(n_steps, features)))
model_gru.add(Dense(64, activation='relu'))
model_gru.add(Dense(forecasting_horizon))
model_gru.compile(optimizer=Adam(0.001), loss='mse', metrics=['mae'])
model_gru.summary()
```

Model: "sequential_21"

Layer (type)	Output Shape	Param #
gru_8 (GRU)	(None, 150)	70,650
dense_26 (Dense)	(None, 64)	9,664
dense_27 (Dense)	(None, 24)	1,560

Total params: 81,874 (319.82 KB)
Trainable params: 81,874 (319.82 KB)
Non-trainable params: 0 (0.00 B)

```
keras_callbacks = [EarlyStopping(monitor='val_loss', patience=20, verbose=1)]
history = model_lstm.fit(
    X_train,
    y_train.reshape(y_train.shape[0], y_train.shape[1]),
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    callbacks=keras_callbacks,
    verbose=2
)
```



600/600 - 5s - loss: 0.0000 - val_loss: 0.0000 - mean: 0.0000 - val_mean: 0.0000 - val_std: 0.0000

```

698/698 - 5s - 7ms/step - loss: 0.0055 - mae: 0.0511 - val_loss: 0.0121 - val_mae: 0.0720
Epoch 8/100
698/698 - 5s - 7ms/step - loss: 0.0055 - mae: 0.0510 - val_loss: 0.0120 - val_mae: 0.0717
Epoch 9/100
698/698 - 5s - 7ms/step - loss: 0.0055 - mae: 0.0508 - val_loss: 0.0120 - val_mae: 0.0715
Epoch 10/100
698/698 - 6s - 8ms/step - loss: 0.0055 - mae: 0.0507 - val_loss: 0.0120 - val_mae: 0.0713
Epoch 11/100
698/698 - 3s - 5ms/step - loss: 0.0055 - mae: 0.0506 - val_loss: 0.0120 - val_mae: 0.0711
Epoch 12/100
698/698 - 3s - 5ms/step - loss: 0.0055 - mae: 0.0505 - val_loss: 0.0119 - val_mae: 0.0709
Epoch 13/100
698/698 - 6s - 8ms/step - loss: 0.0055 - mae: 0.0505 - val_loss: 0.0119 - val_mae: 0.0708
Epoch 14/100
698/698 - 3s - 5ms/step - loss: 0.0055 - mae: 0.0504 - val_loss: 0.0119 - val_mae: 0.0706
Epoch 15/100
698/698 - 6s - 9ms/step - loss: 0.0055 - mae: 0.0503 - val_loss: 0.0119 - val_mae: 0.0705
Epoch 16/100
698/698 - 4s - 6ms/step - loss: 0.0054 - mae: 0.0502 - val_loss: 0.0120 - val_mae: 0.0704
Epoch 17/100
698/698 - 5s - 7ms/step - loss: 0.0054 - mae: 0.0502 - val_loss: 0.0120 - val_mae: 0.0704
Epoch 18/100
698/698 - 4s - 6ms/step - loss: 0.0054 - mae: 0.0501 - val_loss: 0.0120 - val_mae: 0.0703
Epoch 19/100
698/698 - 4s - 6ms/step - loss: 0.0054 - mae: 0.0500 - val_loss: 0.0120 - val_mae: 0.0702
Epoch 20/100
698/698 - 3s - 5ms/step - loss: 0.0054 - mae: 0.0500 - val_loss: 0.0121 - val_mae: 0.0702
Epoch 21/100
698/698 - 5s - 8ms/step - loss: 0.0054 - mae: 0.0499 - val_loss: 0.0122 - val_mae: 0.0703
Epoch 22/100
698/698 - 3s - 5ms/step - loss: 0.0054 - mae: 0.0498 - val_loss: 0.0123 - val_mae: 0.0704
Epoch 23/100
698/698 - 6s - 8ms/step - loss: 0.0054 - mae: 0.0498 - val_loss: 0.0126 - val_mae: 0.0706
Epoch 24/100
698/698 - 3s - 5ms/step - loss: 0.0054 - mae: 0.0497 - val_loss: 0.0128 - val_mae: 0.0708
Epoch 25/100
698/698 - 3s - 5ms/step - loss: 0.0054 - mae: 0.0497 - val_loss: 0.0129 - val_mae: 0.0709
Epoch 26/100
698/698 - 6s - 8ms/step - loss: 0.0053 - mae: 0.0497 - val_loss: 0.0129 - val_mae: 0.0708
Epoch 27/100
698/698 - 4s - 6ms/step - loss: 0.0053 - mae: 0.0496 - val_loss: 0.0130 - val_mae: 0.0708
Epoch 28/100
698/698 - 5s - 7ms/step - loss: 0.0053 - mae: 0.0496 - val_loss: 0.0130 - val_mae: 0.0706
Epoch 29/100
698/698 - 5s - 7ms/step - loss: 0.0053 - mae: 0.0495 - val_loss: 0.0130 - val_mae: 0.0705
Epoch 30/100
698/698 - 5s - 7ms/step - loss: 0.0053 - mae: 0.0494 - val_loss: 0.0129 - val_mae: 0.0703
Epoch 31/100
698/698 - 6s - 8ms/step - loss: 0.0053 - mae: 0.0494 - val_loss: 0.0130 - val_mae: 0.0703
Epoch 32/100
698/698 - 5s - 7ms/step - loss: 0.0053 - mae: 0.0493 - val_loss: 0.0132 - val_mae: 0.0705
Epoch 33/100
698/698 - 6s - 8ms/step - loss: 0.0052 - mae: 0.0492 - val_loss: 0.0134 - val_mae: 0.0706
Epoch 34/100
698/698 - 4s - 6ms/step - loss: 0.0052 - mae: 0.0491 - val_loss: 0.0135 - val_mae: 0.0706
Epoch 34: early stopping

```

RNN testing

```

#Prepare test samples
X_test, y_test = split_sequence(test_set_scaled, n_steps, forecasting_horizon, y_index)
number_test_samples = X_test.shape[0]
print('X_test shape:', X_test.shape)
print('y_test shape:', y_test.shape)

```

```

X_test shape: (10129, 48, 5)
y_test shape: (10129, 24)

```

```

#Predict using trained LSTM
predicted_pm25 = model_lstm.predict(X_test)

```

```

#Inverse scale predictions & true values#
# Only PM2.5 column min/max
pm25_min = sc.data_min_[y_index]
pm25_max = sc.data_max_[y_index]

```

```

predicted_pm25_rescaled = predicted_pm25 * (pm25_max - pm25_min) + pm25_min
y_test_rescaled = y_test * (pm25_max - pm25_min) + pm25_min

```



```
print('predicted_pm25_rescaled shape:', predicted_pm25_rescaled.shape)
print('y_test_rescaled shape:', y_test_rescaled.shape)
```

⇒ 317/317 ————— 1s 2ms/step
predicted_pm25_rescaled shape: (10129, 24)
y_test_rescaled shape: (10129, 24)

```
# Function to compute MAE
def return_mae(test, predicted):
    mae = mean_absolute_error(test, predicted)
    print("Mean Absolute Error {:.2f}.".format(mae))

for i in range(forecasting_horizon):
    print("Forecasting Horizon: {}".format(i+1))
    return_mae(y_test_rescaled[:, i], predicted_pm25_rescaled[:, i])
    print("")
```

⇒

Forecasting Horizon: 6	Mean Absolute Error 34.26.
Forecasting Horizon: 7	Mean Absolute Error 36.36.
Forecasting Horizon: 8	Mean Absolute Error 38.75.
Forecasting Horizon: 9	Mean Absolute Error 40.99.
Forecasting Horizon: 10	Mean Absolute Error 41.64.
Forecasting Horizon: 11	Mean Absolute Error 42.91.
Forecasting Horizon: 12	Mean Absolute Error 44.28.
Forecasting Horizon: 13	Mean Absolute Error 44.75.
Forecasting Horizon: 14	Mean Absolute Error 46.27.
Forecasting Horizon: 15	Mean Absolute Error 47.80.
Forecasting Horizon: 16	Mean Absolute Error 49.06.
Forecasting Horizon: 17	Mean Absolute Error 50.24.
Forecasting Horizon: 18	Mean Absolute Error 50.94.
Forecasting Horizon: 19	Mean Absolute Error 53.40.
Forecasting Horizon: 20	Mean Absolute Error 53.49.
Forecasting Horizon: 21	Mean Absolute Error 56.47.
Forecasting Horizon: 22	Mean Absolute Error 57.88.
Forecasting Horizon: 23	Mean Absolute Error 56.83.
Forecasting Horizon: 24	Mean Absolute Error 58.15.

Inverse transform the test data to original unit for visualization

```
# Inverse transform X_test for visualization
X_test_2d = X_test.reshape(-1, features)
X_test_2d = sc.inverse_transform(X_test_2d)
X_test_rescaled = X_test_2d.reshape(number_test_samples, n_steps, features)
```

Visualize the testing and predicted values of each testing time series segment.

```
sample_index = 6

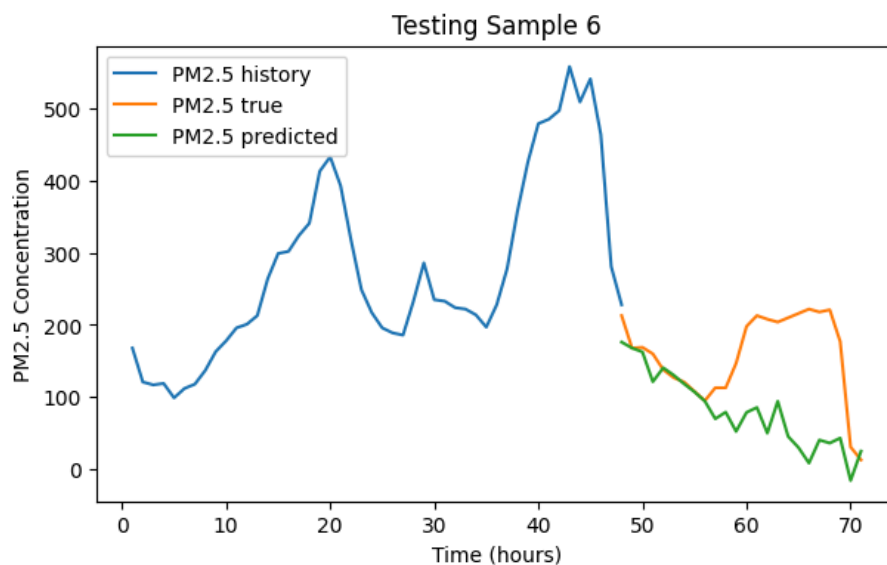
plt.figure(figsize=(7, 4))

# Plot X_test history (PM2.5 history)
plt.plot(np.arange(1, n_steps+1), X_test_rescaled[sample_index, :, 4], label='PM2.5 history')

# Plot true PM2.5 forecast
plt.plot(np.arange(n_steps, n_steps+forecasting_horizon), y_test_rescaled[sample_index, :], label='PM2.5 true')

# Plot predicted PM2.5 forecast
plt.plot(np.arange(n_steps, n_steps+forecasting_horizon), predicted_pm25_rescaled[sample_index, :], label='PM2.5 predicted')

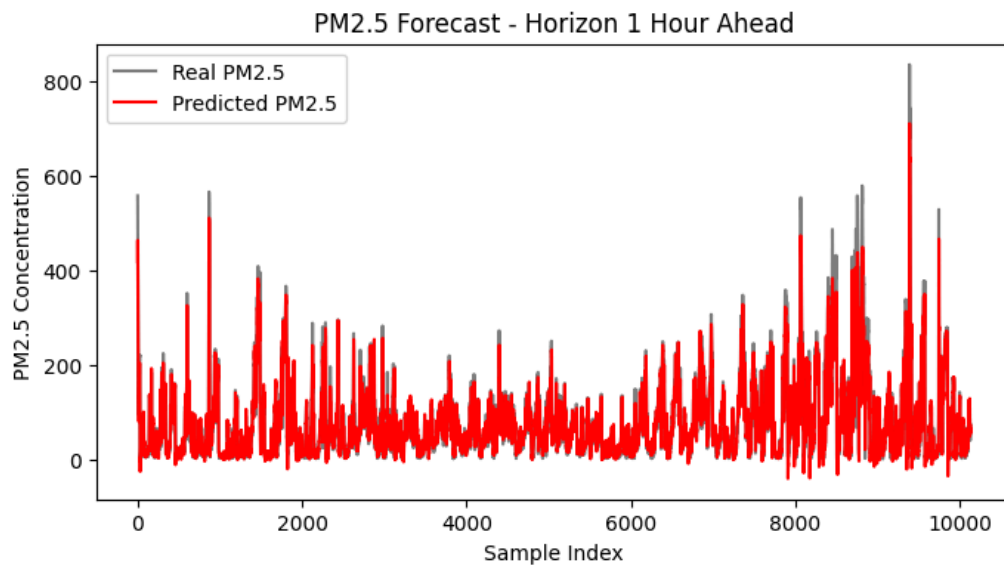
plt.title('Testing Sample ' + str(sample_index))
plt.ylabel('PM2.5 Concentration')
plt.xlabel('Time (hours)')
plt.legend(loc='upper left')
plt.show()
```



◆ Gemini

```
#Function to plot predictions
def plot_predictions(test, predicted, horizon_index=0):
    plt.figure(figsize=(8, 4))
    plt.plot(test[:, horizon_index], color="gray", label="test")
    plt.plot(predicted[:, horizon_index], color="red", label="predicted")
    plt.title(f"PM2.5 Forecast - Horizon {horizon_index}")
    plt.xlabel("Sample Index")
    plt.ylabel("PM2.5 Concentration")
    plt.legend()
    plt.show()
```

```
# Plot for the first horizon across all test samples
plot_predictions(y_test_rescaled, predicted_pm25_rescaled, horizon_index=0)
```



4. Experiments Report

```
import pandas as pd

# Create the data
data = {
    "Iteration": [1, 2, 3, 4],
    "Model Architecture": ["LSTM", "LSTM (Stacked)", "Bidirectional LSTM", "GRU"],
    "Settings": [
        "Hidden Layers: 1; LSTM Units: 100; Dense Units: 24",
        "Hidden Layers: 2; LSTM1 Units: 128; LSTM2 Units: 64; No Dropout; Dense Units: 24",
        "Hidden Layers: 1; LSTM Units: 100 (200 total); Dense1 Units: 64; Dense2 Units: 24",
        "Hidden Layers: 1; GRU Units: 150; Dense1 Units: 64; Dense2 Units: 24"
    ],
    "Average MAE": [43.14, 49.18, 46.8, 43.79]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display as table
df
```



	Iteration	Model Architecture	Settings	Average MAE	
0	1	LSTM	Hidden Layers: 1; LSTM Units: 100; Dense Units...	43.14	
1	2	LSTM (Stacked)	Hidden Layers: 2; LSTM1 Units: 128; LSTM2 Unit...	49.18	
2	3	Bidirectional LSTM	Hidden Layers: 1; LSTM Units: 100 (200 total);...	46.80	
3	4	GRU	Hidden Layers: 1; GRU Units: 150; Dense1 Units...	43.79	

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

*Interpretation of Results

Simple LSTM performed best (MAE = 43.14)

Indicates that a single-layer LSTM is sufficient to capture temporal dependencies in PM2.5 data.

Adding complexity (stacked LSTM or Bidirectional LSTM) increased errors, likely due to overfitting or difficulty learning from noisy hourly data.

Stacked LSTM had the worst performance (MAE = 49.18)

Extra layers did not improve forecasting; they may have caused the network to memorize noise rather than general patterns.

GRU (MAE = 43.79)

Slightly worse than simple LSTM, but still competitive.

GRU trains faster, so could be a good alternative when computational speed is a concern.

Bidirectional LSTM (MAE = 46.8)

Did not improve over single LSTM; may be less suitable for hourly PM2.5 forecasting due to noisy forward/backward sequences.*

Meaning of the Results

Simpler architectures work better for noisy, multivariate hourly time series.

Overly complex models tend to overfit and fail to generalize.

The single-layer LSTM balances complexity and generalization, providing the lowest prediction error for a 24-hour PM2.5 forecast.

Business Application

Air Quality Forecasting

The model predicts PM2.5 concentrations 24 hours in advance using key pollutants (CO, NO2, SO2, WSPM).

Helps municipalities and environmental agencies issue early health advisories.

Public Health & Policy

Accurate forecasts enable preventive measures, such as reducing industrial emissions or advising citizens with respiratory conditions.

Urban Planning

Long-term modeling of pollution trends can inform transportation planning, green space design, and industrial zoning.

✓ 5. Role of GenAI

Generative AI (ChatGPT) was used as a collaborative tool throughout the PM2.5 forecasting project. It assisted in designing and testing different RNN architectures, including LSTM, GRU, and Bidirectional LSTM, by suggesting optimal layer configurations, activation functions, and sequence lengths for multivariate time series. GenAI provided guidance on data preprocessing, scaling, reshaping, and early stopping, helping troubleshoot coding errors and refine the training workflow.

It also supported interpretation of experimental results, enabling comparisons of model performance and identification of the most accurate architecture. Despite these advantages, using GenAI required careful oversight, as suggestions needed validation against the dataset to ensure stable and meaningful predictions. Ensuring model generalization and avoiding overfitting were the key challenges in leveraging GenAI effectively.